

ROS2 Spring Workshop

Lab 8: Hardware Deployment & Calibration

Objective

For 2.5 weeks, you have programmed in a simulated environment where motors never stall, wheels never slip, and gravity is constant. Today, we execute our hardware deployment. You will move your code onto the physical Raspberry Pi, verify the hardware interfaces, and confront the **Reality Gap**.

How This Lab Fits the AgTech Triathlon

Your U-turn challenge today — drive 3 meters, turn around, drive 3 meters back — is not arbitrary. It is a direct primitive for the Final Project: it proves your robot can drive a row, execute a headland turn, and drive the next row. Every part of the zigzag field pattern is built on this single maneuver.

Pre-Lab Concept Primer

Before we connect to the hardware, you must understand the core concepts we are using to bridge the gap between your laptop and the physical robot.

1. SSH (Secure Shell): Remote Access

Because the Raspberry Pi on the tractor is “headless” (no monitor or keyboard), we must control it over Wi-Fi. We use **SSH** to securely open a text-based terminal inside the robot’s operating system.

2. The Dual-Brain Architecture (Host vs. Docker)

While the robot’s base operating system *does* have a hidden ROS 2 environment used for background tasks, we will not use it for our development. Instead, your custom code will run inside a **Docker container** on your laptop (a lightweight, isolated virtual environment). This ensures you have a clean, standard ROS 2 Humble workspace.

For this lab and all labs after it: your code runs on your laptop’s container. The robot runs the motor driver and publishes odometry. The two machines talk over Wi-Fi via ROS 2 DDS.

3. VS Code Remote Development (The Invisible Cable)

When you want to inspect files *on* the robot (rarely needed in this course), VS Code’s Remote-SSH extension lets you open folders on the Pi as if they were local. Most of the time you’ll just stay in your laptop’s IDE because the code lives there.

4. Ackermann Kinematics: The Tractor Constraint

In TurtleSim, the robot uses differential drive; it can spin in place. Real tractors (and our MentorPi A1) use Ackermann steering, meaning the front wheels pivot while the rear wheels drive. **You cannot change your heading without moving forward.** Every time you send an `angular.z` command with `linear.x` at zero, nothing happens.

5. Motor Encoders & Odometry: Blind Math

The physical robot does not inherently know where it is in the room. Instead, it uses **motor encoders** — physical sensors on the wheel axles that count rotation “ticks.” The low-level controller translates these ticks into a best-guess distance traveled, publishing it to the `/odom` topic. We will use this data to drive precise distances without relying on a stopwatch.

6. Native Control & Hardware Verification

Unlike closed-system robots, the MentorPi control stack accepts standard ROS 2 `/cmd_vel` messages, which are translated by the onboard control software into motor actuation. Before writing any autonomous scripts, we use standard ROS 2 GUI tools (`rqt_robot_steering`) to inject mathematical velocities directly into the system, verifying that the Wi-Fi, the Docker container, and the physical motors are communicating correctly.

Part 0: The Network Handshake (Pre-Configured)

Notice: Instructor Pre-Configuration

You DO NOT need to execute the steps in Part 0. Your lab instructor has already unlocked the 5GHz antennas, locked in static IP addresses for every robot in the fleet, and patched the robot’s Wi-Fi startup sequence to prevent DDS from latching onto the wrong network interface. This section is provided purely so you understand the underlying engineering process of deploying headless hardware on a new network.

Behind the Scenes: Step 1 — The Ethernet Lifeline & Nmap Sweep

When a headless Raspberry Pi is first deployed, it has no monitor and is not connected to Wi-Fi. It must be plugged into a physical Ethernet switch. Because the router assigns it a random, temporary IP address, the instructor must find it on the local network by running a network sweep.

```
1 nmap -sn 192.168.0.0/24
```

Decoding the Command: Nmap

`nmap` is a powerful network scanning tool used to discover devices.

- `-sn`: This flag stands for “Ping Scan.” It tells Nmap not to interrogate specific open ports, but simply to ping each address to see if it is online. It is the fastest way to map a network.
- `192.168.0.0/24`: This is the target IP range. The `/24` is CIDR notation, telling the tool to scan all 256 possible addresses from `192.168.0.0` to `192.168.0.255`.

Behind the Scenes: Step 2 — Unlocking the 5GHz Antenna

By default, the Raspberry Pi’s 5GHz antenna is disabled at the firmware level to comply with international aviation radar laws. The instructor runs `raspi-config`, navigates to **Localisation Options** → **WLAN Country**, and sets it to **CA Canada**.

Behind the Scenes: Step 3 — DHCP Reservation & Power Saving

In a lab environment with multiple robots, we use **DHCP Address Reservation** on the router to prevent IP conflicts. The instructor also disables Wi-Fi power saving on the Pi so the robot does not go dormant and lose its DDS peers:

```
1 sudo iw wlan0 set power_save off
```

Behind the Scenes: Step 4 — DDS Interface Binding

By default, the ROS 2 daemon will bind to the first network interface it finds. On a Pi running Docker, this is often `docker0` (the internal Docker bridge, which goes nowhere). The instructor patches the MentorPi startup service to explicitly bind DDS to `wlan0` and adds a short delay so Wi-Fi is fully up before ROS starts. This is why your laptop container can see `/odom` reliably.

Your Action Required: Your lab instructor will tell you your assigned robot IP. Write it down on your lab worksheet now; you will use it repeatedly throughout the lab.

Part 1: The Access Toolkit (The 3 Methods)

About These Methods

You will mostly use **Method A (SSH)** for quick checks and **your laptop's local Docker container** for running your code. Methods B and C are here for when you want a graphical view into the robot or need to edit files that live on the Pi itself — useful, but not required for this lab.

Method A: Standard Terminal SSH (Quick Commands)

1. Connect via SSH: `ssh pi@<your_robot_ip>`
2. Password is `raspberrypi`.

Method B: VNC (Graphical Desktop)

1. Open your VNC Viewer application on your laptop.
2. Enter your robot's IP address.
3. Log in using the same credentials.

Method C: VS Code Remote & Dev Containers (Writing Code On The Pi)

1. Open **VS Code** on your laptop.
2. Install **Remote - SSH** and **Dev Containers** extensions.
3. Press `Ctrl+Shift+P` → **Remote-SSH: Connect to Host...** → enter `pi@<your_robot_ip>`.

Part 2: Environment Setup

Important: One Variable Per Session

You only need to set `ROS_DOMAIN_ID` in each new terminal. The robot's boot environment handles everything else automatically.

Do not manually source `.typerc` or set `CYCLONEDDS_URI` in your interactive shell — the robot's startup system manages these correctly. (`.typerc` is a Hiwonder-specific config file; leave it alone.)

Laptop Container Shell

In the dev container in VS Code set the domain:

```
1 export ROS_DOMAIN_ID=<your_robot_number>
2 echo $ROS_DOMAIN_ID
```

Boot Timing Warning

After powering on the robot, wait until the boot process is fully complete and the motor controller emits its startup beep before attempting teleoperation or ROS communication.

If you try to connect too early, the robot may appear unresponsive even when your setup is correct.

Verify topics are visible:

```
1 ros2 topic list
```

You should see a long list including `/odom`, `/cmd_vel`, `/ros_robot_controller/joy`, and `/tf`. If the list looks sparse, wait 10–15 seconds and try again. The ROS 2 daemon needs a moment to discover the network via DDS after a fresh shell.

If Topics Are Still Missing After 30 Seconds

Sometimes the background daemon caches an old `ROS_DOMAIN_ID` or network interface and fails to see new topics. You can bypass the daemon entirely to check the true network state:

```
1 ros2 topic list --no-daemon
```

If this reveals your missing topics, your daemon is stuck on old settings. Fix it permanently by restarting the daemon:

```
1 ros2 daemon stop
```

The daemon will automatically restart with the correct settings the next time you run a ROS 2 command.

Part 3: Controller Teleoperation & Pairing

We are using wireless PS1-style gamepads to manually drive the tractor. In Lab 10 you will use these gamepads as the “deadman switch” for safety. Today you will just use them for fun, this is your first chance to actually drive the thing.

1. **Match Your Hardware:** Ensure your USB receiver and controller both have your Group Number taped to them. Plug the USB receiver into the Pi.
2. **Turn on the Controller:** Flip the power switch on the gamepad.
3. **CRITICAL STEP, RED MODE:** Look at the **MODE LED** on top of the controller.
 - It **MUST** be solid **RED**.
 - If the light is **GREEN**, press and hold the **Mode** button for 5 seconds to switch it.

Why Red Mode Matters

The PS1-style gamepads have two modes. They have different key mappings and ROS node that interprets it knows the RED mapping.

Part 4: Hardware Verification & First Drive

Before writing autonomous code, verify that your laptop container can actually talk to the physical robot. There are **two independent ways to send drive commands**, and you should try both. They prove different things.

The Two Drive Paths

Path A: The ROS GUI (`rqt_robot_steering`) A graphical slider that runs in your laptop container and publishes `/cmd_vel` over Wi-Fi to the robot. This proves your laptop container can reach the robot's motor driver across the network.

Path B: The USB Gamepad A PS1-style controller plugged into the Pi itself. The robot's onboard driver reads the USB input and turns it directly into motor commands. This proves the robot's local hardware stack works end-to-end, no laptop required.

Both end up publishing the same `/cmd_vel` topic, just from different sources. Try Path A first (it's the simpler test), then Path B (it's the more fun test).

Path A: The ROS GUI (“Zero-Code” Test)

This verifies the network path: laptop container → Wi-Fi → Pi → motors.

Industry Insight: The Power of Open Source ROS

One of the best things about ROS is that you rarely have to reinvent the wheel. Need a steering GUI? The community has already built one, open-sourced it on GitHub, and documented how it works:

https://github.com/ros-visualization/rqt_robot_steering.

If you need a standard tool, chances are you can pull it from a community repo and run it natively.

1. **Install the GUI tool** in your **laptop's Docker container** (not the robot):

```
1 apt update
2 apt install ros-humble-rqt-robot-steering -y
3
```

2. **Set your domain.** The variable persists within this terminal, but *every new terminal* needs it declared again:

```
1 export ROS_DOMAIN_ID=<your_robot_number>
2 echo $ROS_DOMAIN_ID
3
```

3. **Allow graphical output** (Linux users only). The GUI is a window, so the container needs permission to draw on your screen:

```
1 xhost +local:
2 export DISPLAY=:1
3
```

4. **Launch the tool:**

```
1 ros2 run rqt_robot_steering rqt_robot_steering
2
```

5. A graphical window will appear. **Place the robot on blocks with wheels free before touching the sliders.** Slowly drag the vertical speed slider upwards. The physical tractor wheels should spin forward.

Why No `colcon` Build?

When you install a ROS tool with `apt install`, it goes straight into `/opt/ros/humble/` as a pre-built binary; no compilation needed. `colcon` is only for **your own code** that you write from scratch and need to register as a ROS node. Community tools like `rqt_robot_steering` are already built and packaged, so `apt` just drops them in ready to run.

Path B: The USB Gamepad (Drive It Around)

Now the fun part. The motor driver running on the Pi is already reading the gamepad USB receiver directly. No laptop involved in the loop; your controller talks to the robot's onboard driver, which publishes `/cmd_vel` locally.

With the controller paired (Part 3) and in **red mode**, take the robot off the blocks and put it on the floor:

1. Keep the robot in an open space. 2–3 metres of clearance all around.
2. Watch the motor controller LEDs. They should flicker when you move the sticks.
3. Drive the robot around for a minute. Get a feel for:
 - How fast it actually moves compared to what you expected.
 - How tight it can turn.
 - What happens when you try to turn while stopped (spoiler: nothing, Ackermann).
 - How quickly it responds to your inputs.

Two Paths Using One Topic

If you run `ros2 topic echo /cmd_vel` while driving with the gamepad, you'll see exactly the same kind of messages the GUI was producing. The robot doesn't care *who* is publishing, only what shows up on the topic. This is the core of ROS: every input source speaks the same language, and you can swap one for another without changing the motor driver. In Lab 10 you'll use this same principle to plug an autonomous controller into the same `/cmd_vel` stream.

Safety Note: How To Stop The Robot Fast

If the robot misbehaves at any point during this lab:

1. Release all joystick sticks. Manual commands should stop within milliseconds.
2. If autonomous code is running, press Ctrl+C in the terminal.
3. If Ctrl+C doesn't stop it, **pick up the robot**. The wheels can spin freely in the air without damage.
4. As a last resort, power cycle the robot.

Note: Ctrl+C kills your code, but the motor driver on the Pi may hold the last `/cmd_vel` command briefly (up to ~500 ms) before timing out. Always be ready to physically lift the robot.

Part 5: Closed-Loop Control (The Odometry Challenge)

Your final challenge is a precise 3-meter out-and-back mission using an autonomous state machine running on your **laptop container**. The `/odom` and `/cmd_vel` topics are bridged over Wi-Fi, so your laptop node can subscribe to robot odometry and publish motion commands exactly as if it were running on the robot itself.

This U-turn primitive is the foundation of the AgTech Triathlon: drive a row, turn at the headland, drive the next row.

Step 1: Package Setup

In your laptop container, create the package:

```
1 export ROS_DOMAIN_ID=<your_robot_number>
2 cd /workspaces/agtech_ros2
3 ros2 pkg create --build-type ament_python src/lab8_hardware
```

Step 2: Code Analysis & The Skeleton Driver

Below is the structure of the `base_driver.py` file. Currently, it drives blindly for one second. You will replace the blind timer with an odometry-based state machine.

Reality Gap: Why Odometry Lies

The `/odom` topic on this robot is produced by `ekf_filter_node`, which fuses wheel encoder data, IMU, and RF2O laser odometry into a single state estimate. In practice you will observe two important limitations:

Position drift: The X/Y position estimate is reasonably accurate over short distances but accumulates error over time as wheel slip and encoder noise compound. This is why your robot may not return to exactly its starting point.

Yaw overcounting: The IMU on this hardware drifts significantly, and the EKF gains are not tuned for this specific robot. Small physical rotations get reported as much larger angular changes. In testing, a 45-degree physical turn was reported as over 2.3 radians (roughly 135 degrees). This makes yaw-based turn detection unreliable, which is why we will use a calibrated timer for the turn instead of trusting the reported yaw.

This is the **Reality Gap** in action. Real robots do not have perfect sensors, and a significant part of robotics engineering is understanding the limits of your state estimate and designing controllers that are robust to those limits. Your calibration deliverable asks you to measure and document exactly this phenomenon.

- Create `base_driver.py` inside `lab8_hardware/lab8_hardware/`.
- The code below is also in the lab resource folder; complete all TODO blocks.

```
1 import rclpy
2 from rclpy.node import Node
3 from geometry_msgs.msg import Twist
4 from nav_msgs.msg import Odometry
5 import math
6
7 class AutonomousDriver(Node):
8     def __init__(self):
9         super().__init__('autonomous_driver')
10        self.publisher_ = self.create_publisher(Twist, '/cmd_vel', 10)
11        self.subscription = self.create_subscription(
12            Odometry, '/odom', self.odom_callback, 10)
13        self.start_time = self.get_clock().now()
14        self.current_x = 0.0
15        self.current_y = 0.0
16        self.current_yaw = 0.0
17        # TODO 1: Add variables for your state machine
18        #     e.g., self.state = 'FORWARD_1'
19        # TODO 2: Add variables to store starting X, Y, and Yaw.
20        #     Also add: self.turn_start_time = None
21        self.timer = self.create_timer(0.1, self.control_loop)
22
23    def get_yaw(self, q):
24        # Converts 4D Quaternion to 1D Euler Angle (Yaw)
25        siny_cosp = 2 * (q.w * q.z + q.x * q.y)
26        cosy_cosp = 1 - 2 * (q.y * q.y + q.z * q.z)
27        return math.atan2(siny_cosp, cosy_cosp)
28
29    def odom_callback(self, msg):
30        # TODO 3: Extract X and Y positions from the Odometry message
31        #     Hint: msg.pose.pose.position.x
32        # TODO 4: Extract orientation and pass it to self.get_yaw()
33        pass
34
35    def control_loop(self):
36        msg = Twist()
37        # =====
38        # CURRENT BEHAVIOUR: 1-SECOND BLIND TIMER DRIVE
39        # =====
40        elapsed = (self.get_clock().now() - self.start_time).nanoseconds / 1e9
```

```

41         if elapsed < 1.0:
42             msg.linear.x = 0.15
43         else:
44             msg.linear.x = 0.0
45         # =====
46         # CHALLENGE: REPLACE THE ABOVE WITH YOUR STATE MACHINE
47         # =====
48         # TODO 5: Calculate Euclidean distance from start position
49         # TODO 6: Build the State Machine:
50         #     IF state == 'FORWARD_1':
51         #         Drive until distance >= 3.0 meters. Switch to 'TURN'.
52         #     ELIF state == 'TURN':
53         #         Use a timer to control turn duration (yaw is unreliable!)
54         #         Starting values to try: linear.x = 0.10, angular.z = 0.5,
55         #         duration ~6 seconds for a 180 degree turn. Tune from there.
56         #         Initialize self.turn_start_time when entering TURN, then check:
57         #         turn_elapsed = (self.get_clock().now()
58         #             - self.turn_start_time).nanoseconds / 1e9
59         #         REMINDER: Ackermann steering requires linear.x > 0 to turn!
60         #     ELIF state == 'FORWARD_2':
61         #         Drive back until distance from turn-point >= 3.0 meters.
62         #         Switch to 'DONE'.
63         #     ELIF state == 'DONE':
64         #         Stop motors.
65         # HARDWARE SAFETY CLAMP (DO NOT REMOVE)
66         if msg.angular.z > 0.5:
67             msg.angular.z = 0.5
68         elif msg.angular.z < -0.5:
69             msg.angular.z = -0.5
70         self.publisher_.publish(msg)
71
72 def main(args=None):
73     rclpy.init(args=args)
74     node = AutonomousDriver()
75     try:
76         rclpy.spin(node)
77     except KeyboardInterrupt:
78         pass
79     finally:
80         # SAFETY: publish zero velocity on shutdown so the robot
81         # doesn't coast after Ctrl+C. Reuse this pattern in Labs 9, 10.
82         stop_msg = Twist()
83         node.publisher_.publish(stop_msg)
84         node.destroy_node()
85         rclpy.shutdown()
86
87 if __name__ == '__main__':
88     main()

```

Code Walkthrough: What Each Block Does

This script wires together sensors, math, and motors, but the structure is straightforward once you know what each block is for. Read this once before filling in the TODOs.

- **Imports:** The script pulls in `rclpy` (the ROS 2 Python library), `Node` (the base class your driver inherits from), `Twist` (the message type for motion commands, containing linear and angular velocity), `Odometry` (the message type the robot publishes on `/odom` with its pose estimate), and `math` (for the quaternion conversion and Euclidean distance).
- **Node Setup (`__init__`):** The constructor creates one publisher to `/cmd_vel` and one subscriber to `/odom`. It also initialises three pose variables (`current_x`, `current_y`, `current_yaw`) that the callback will keep up to date, and records `start_time` for the placeholder blind-timer behaviour. Finally, it starts a timer that fires the `control_loop` function every 0.1 seconds (10 Hz).
- **The Control Loop Timer:** 10 Hz means the robot makes a fresh decision ten times per second. This is fast enough that it can react smoothly to changing odometry, but slow enough that you don't drown the network in messages. Real autonomous systems run their control loops at fixed rates for exactly this reason.

- **The Quaternion-to-Yaw Helper (`get_yaw`):** ROS publishes orientation as a 4D quaternion (x, y, z, w) because four numbers can represent any 3D rotation cleanly and without edge cases, whereas three angles (roll, pitch, yaw) hit weird math problems at certain orientations. For a robot driving on a flat floor, none of that matters; the only rotation we care about is yaw. This helper does the conversion math for you. You don't need to memorise it; you just need to know it returns the robot's heading in radians.
- **The Odometry Callback (`odom_callback`):** Every time the robot publishes a new `/odom` message (roughly 20–30 times per second), this function runs. Your job in TODO 3 and TODO 4 is to extract the X and Y position and the yaw from the message, and store them in the instance variables so the control loop can read them.
- **Current Behaviour (Blind Timer Drive):** The code currently checks how many seconds have elapsed since startup, drives at 0.15 m/s for the first second, then stops. This is exactly the “guess and hope” approach Lab 3 used. It ignores odometry completely. Your challenge is to replace it with sensor-driven logic.
- **The Challenge Zone (State Machine):** TODOs 5 and 6 are the main exercise. You will calculate Euclidean distance from your starting point and use it to decide which state you are in: driving out, turning around, driving back, or done. The turn state uses a wall-clock timer instead of yaw because (as the Reality Gap box explains) the IMU on this hardware overcounts angular rotation badly enough that a yaw-based check would fire far too early.
- **The Safety Clamp:** Right before publishing, the code clamps `angular.z` to the range $[-0.5, +0.5]$. This is a hardware safety limit. If your state machine accidentally calculates a huge angular velocity (for example, because you forgot to normalise an angle), the clamp prevents the robot from trying to execute a violent turn. Do not remove it.
- **Publishing the Twist:** The final line of every control loop iteration sends the `msg` object to `/cmd_vel`, where the motor driver picks it up and converts it into wheel speeds and a steering angle. If you forget this line, the robot does nothing regardless of how clever your state machine is.
- **Safe Shutdown (`main`):** The `try/finally` pattern ensures that when you press Ctrl+C, the code catches the interrupt, then publishes a zero Twist before shutting down. Without this, the robot would coast on whatever its last command was for up to 500 ms before the motor driver's watchdog kicked in. Always include this in any node that publishes `/cmd_vel`.

Ackermann Steering Hint

The MentorPi uses Ackermann steering and **cannot change heading without moving forward**. During your TURN state, make sure `linear.x` is greater than zero, otherwise the robot will sit perfectly still and never complete the turn.

Why The try/finally Block Matters

Look at the `main()` function carefully. When you press Ctrl+C, the `except KeyboardInterrupt` catches it, and the `finally` block publishes a zero-velocity `Twist` before the node shuts down. **Without this, the robot would keep driving with its last command until the motor driver's safety timeout kicks in** (up to ~500 ms).

This pattern, “always leave the robot stopped when your code exits”, is a core habit of professional robotics engineers. You will reuse it in Lab 9 and the Final Project.

Step 3: Register and Build

Add `base_driver` to your `setup.py` entry points:

```
1 entry_points={
2     'console_scripts': [
3         'base_driver = lab8_hardware.base_driver:main',
4     ],
5 }
```

Then build:

```
1 cd /workspaces/agtech_ros2
2 colcon build --packages-select lab8_hardware
3 source install/setup.bash
```

Step 4: Execute

1. **Safety check first:** Robot on blocks, wheels free. Do not place on the floor until Step 5.
2. **Initial Test:** Run `ros2 run lab8_hardware base_driver`. Notice that it currently ignores the sensors and drives forward for 1 second blindly.
3. **The Odometry Upgrade:** Complete the TODO blocks. Replace the 1-second timer with your state machine to drive 3 meters, perform a 180-degree turn, and return.
4. **Tune the turn:** Your first attempt probably won't hit exactly 180 degrees. Tape two markers 3 m apart on the floor, run the mission, and adjust the turn duration until the robot ends up reasonably close to its starting point.

```
1 ros2 run lab8_hardware base_driver
```

Step 5: Floor Test

Only once the state machine works on blocks (wheels spin through the right sequence) should you move to the floor. Choose a 7-meter stretch of clear space (3 m out + ~1 m for the turn arc + 3 m back).

Reflection Question (Discuss With Your Group)

What happens if your robot drives into a wall during FORWARD_1? The wheels keep spinning against the floor, so the encoder ticks keep accumulating. Your state machine thinks the robot has driven 3 meters even though it's been stuck against the wall for 15 seconds.

How would you detect this? You don't need to implement a solution; just describe one. (Hints: compare commanded vs. achieved velocity, use the LIDAR, watch motor current, or use a bump sensor.)

Deliverables

Submit a PDF containing:

1. **Calibration Data:** Run your FORWARD_1 (3-meter straight drive) **five times**, with the robot's starting position marked by tape on the floor. After each run, tape-measure the actual distance traveled. Report:
 - A table of the five measurements.
 - The mean and standard deviation.
 - The percentage drift: $\left| \frac{\text{mean_actual} - 3.0}{3.0} \right| \times 100\%$.
 - A one-sentence interpretation: is the robot consistently short, consistently long, or just noisy?

Drift under 5% means your odometry is well-calibrated for this scale. Drift over 15% suggests wheel slip or miscalibrated encoders — note in your writeup.

2. **Challenge Proof:** A picture your robot “moving” around.
3. **Reflection Answer:** In 2–3 sentences, answer the wall-collision question from Step 5. Which detection approach would you actually implement, and why?